LAB PROGRAM 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

b) Earliest-deadline First

c) Proportional scheduling

a) Rate- Monotonic

CODE:

```
#include <stdio.h>
#include <math.h>

//TO CALCULATE GCD
int gcd(int a, int b)
{
   return (b == 0) ? a : gcd(b, a % b);
}

//TO CALCULATE LCM
int lcm(int a, int b)
{
   return (a * b) / gcd(a, b);
}

int main()
```

```
{
    int n, i;
    int bt[10], pt[10], rt[10];
    int hyper, time = 0;

    printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst Times:\n");
    for (i = 0; i < n; i++)
        {
            scanf("%d", &bt[i]);
            rt[i] = 0; // initially no job released
        }

    printf("Enter Periods:\n");
    for (i = 0; i < n; i++)
        {
            scanf("%d", &pt[i]);
        }

// Calculate hyperperiod = LCM of all periods
    hyper = pt[0];
    for (i = 1; i < n; i++)
```

```
        {
            hyper = lcm(hyper, pt[i]);
        }

    printf("LCM=%d\n\n", hyper);

    printf("Rate Monotone Scheduling:\n");
    printf("PID   Burst   Period\n");
    for (i = 0; i < n; i++)
        {
            printf("%d     %d      %d\n", i+1, bt[i], pt[i]);
        }

// Utilization check
    double U = 0;
    for (i = 0; i < n; i++)
        {
            U += (double)bt[i] / pt[i];
        }
    double bound = n * (pow(2.0, 1.0/n) - 1);

    printf("\n%.6f <= %.6f => %s\n", U, bound, (U <= bound) ? "true" : "false");
    printf("Scheduling occurs for %d ms\n\n", hyper);

// Timeline simulation (block-style)
    int current_running = -1;
```

```
    while (time < hyper)
        {
// Release jobs
        for (i = 0; i < n; i++) {
            if (time % pt[i] == 0) {
                rt[i] = bt[i];
            }
        }

// Pick highest priority (smallest period)
        int min_period = 9999, index = -1;
        for (i = 0; i < n; i++) {
            if (rt[i] > 0 && pt[i] < min_period) {
                min_period = pt[i];
                index = i;
            }
        }

        // Print only when event changes
        if (index != current_running)
            {
            if (index != -1)
            {
                printf("%dms onwards: Process %d running\n", time, index+1);
            }
        else
```

```
        {
            printf("%dms onwards: CPU is idle\n", time);
        }
        current_running = index;
        }

    // Execute one unit if a process is running
    if (index != -1)
        {
            rt[index]--;
        }

    time++;
  }

  return 0;
}
```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W

        NAME : R K MADAN        USN : 1WA24CS225        SECTION : 'O'

Enter number of processes: 2
Enter Burst Times:
20 35
Enter Periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID   Burst   Period
1     20      50
2     35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 14.917 s
Press any key to continue.
```

b) Earliest-deadline First

CODE:

```
#include <stdio.h>


int main()
{
   int n, i;


   printf("\n\tNAME : R K MADAN\tUSN : 1WA24CS225\tSECTION : 'O'\n\n");
   printf("Enter number of tasks: ");
   scanf("%d", &n);
   int bt[n], dl[n], p[n], rem[n];
   printf("Enter Burst times:\n");
```

```
for (i = 0; i < n; i++) {
    scanf("%d", &bt[i]);
    rem[i] = 0; // initially no job released
}

printf("Enter Deadlines:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &dl[i]);
}

printf("Enter Periods:\n");
for (i = 0; i < n; i++) {
    scanf("%d", &p[i]);
}

printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t\t%d\n", i + 1, bt[i], dl[i], p[i]);
}

int total_time = p[0];
for (i = 1; i < n; i++) {
    if (p[i] > total_time)
        total_time = p[i];
}
```

```
    printf("\nScheduling occurs for %d ms\n\n", total_time);

    int prev_idle = 0;



    for (int time = 0; time < total_time; time++) {
        for (i = 0; i < n; i++) {
            if (time % p[i] == 0) {
                rem[i] = bt[i];
            }
        }

// Select task with earliest deadline
        int selected = -1;
        int min_deadline = 9999;
        for (i = 0; i < n; i++) {
            if (rem[i] > 0) {
                if (dl[i] < min_deadline) {
                    min_deadline = dl[i];
                    selected = i;
                }
            }
        }

// Print timeline
        if (selected == -1) {
```

```
            if (!prev_idle) {
                printf("%dms onwards: CPU is idle.\n", time);
                prev_idle = 1;
            }
        } else {
            printf("%dms onwards: Task %d is running.\n", time, selected + 1);
            rem[selected]--;
            prev_idle = 0;
        }
    }

    return 0;
}
```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W

PID     Burst   Deadline        Period
1       3       7               20
2       2       4               5
3       2       8               10

Scheduling occurs for 20 ms

0ms onwards: Task 2 is running.
1ms onwards: Task 2 is running.
2ms onwards: Task 1 is running.
3ms onwards: Task 1 is running.
4ms onwards: Task 1 is running.
5ms onwards: Task 2 is running.
6ms onwards: Task 2 is running.
7ms onwards: Task 3 is running.
8ms onwards: Task 3 is running.
9ms onwards: CPU is idle.
10ms onwards: Task 2 is running.
11ms onwards: Task 2 is running.
12ms onwards: Task 3 is running.
13ms onwards: Task 3 is running.
14ms onwards: CPU is idle.
15ms onwards: Task 2 is running.
16ms onwards: Task 2 is running.
17ms onwards: CPU is idle.

Process returned 0 (0x0)   execution time : 33.521 s
Press any key to continue.
```

c) Proportional scheduling

CODE:

```
// Proportional Share Scheduling (Lottery Scheduling)


#include <stdio.h>

#include <stdlib.h>

#include <time.h>


// Define Process structure

typedef struct {

   char name[5];

   int tickets;
```

```
} Process;

int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes: ");
    scanf("%d", &n);

    Process p[n];
    srand(time(NULL)); // Seed random number generator

    // Input process details
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);

        printf("Tickets: ");
        scanf("%d", &p[i].tickets);

        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }

    printf("\n--- Proportional Share Scheduling ---\n");
```

```
// Input scheduling time period
int m;
printf("Enter the Time Period for scheduling: ");
scanf("%d", &m);

// Lottery scheduling simulation
for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index = -1;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets picked: %d, Winner: %s\n",
           winning_ticket, p[winner_index].name);
}

// Print proportional share of CPU time
for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %.2f%% of Processor Time.\n",
```

```
            p[i].name, ((p[i].tickets / total_T) * 100));
    }

    return 0;
}
```

OUTPUT:

```
"C:\Users\Admin\Desktop\1W

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 56, Winner: P3
Tickets picked: 10, Winner: P1
Tickets picked: 5, Winner: P1
Tickets picked: 34, Winner: P3
Tickets picked: 38, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 13.316 s
Press any key to continue.
```